

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Programación asíncrona



Universidad
Rey Juan Carlos

Programación asíncrona en Android

Las aplicaciones usan un subproceso principal para controlar las interacciones con la UI. **Lanzar tareas de larga duración desde este subproceso puede generar bloqueos y falta de respuesta.**

La programación asíncrona nos permite lidiar con múltiples requisitos de las apps móviles:

- Tiempo limitado para realizar tareas y **mantener la capacidad de respuesta**
- Equilibrio con la necesidad de **ejecutar tareas de larga duración**
- **Control sobre cómo y dónde** se ejecutan las tareas

Ejemplos de tareas de larga duración: obtener o sincronizar datos con un servidor remoto, leer o escribir en una base de datos o un archivo, un cálculo pesado...

Coroutines

Coroutines o corrutinas

Una *coroutine* es un **patrón de diseño de concurrencia** disponible en Android para simplificar el código que se ejecuta de forma asíncrona. Las coroutines se añadieron a Kotlin en la versión 1.3.

Las coroutines permiten:

- Mantener la capacidad de respuesta de tu aplicación mientras se gestionan tareas de larga duración
- Simplificar el código asíncrono
- Escribir código de forma secuencial
- Manejar las excepciones con el bloque try/catch

Beneficios de las corrutinas

- **Ligereza:** se pueden ejecutar muchas corrutinas en un solo subproceso debido a la compatibilidad con la suspensión, que no bloquea el subproceso en el que se ejecuta la corrutina. La suspensión ahorra más memoria que el bloqueo y admite muchas operaciones simultáneas.
- **Menos fugas de memoria:** usa la simultaneidad estructurada para ejecutar operaciones dentro de un alcance.
- **Compatibilidad con cancelación integrada:** la cancelación se propaga automáticamente a través de la jerarquía de corrutinas en ejecución.
- **Integración con Jetpack:** muchas bibliotecas Jetpack incluyen extensiones que proporcionan compatibilidad total con corrutinas.

Dependencias de *coroutines*

Para utilizar corrutinas hay que añadir las dependencias necesarias en el archivo *build.gradle.kts (:app)*

```
dependencies {  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.3.9")  
    ...  
}
```

build.gradle.kts (:app)

Funciones para corrutinas (suspend)

Utilice el modificador ***suspend*** para declarar una función como disponible para las **coroutines**. La función sólo se puede invocar **desde dentro de una coroutine o desde otra función de suspensión**.

```
class LibrosRepository(private val dao: LibrosDAO) {  
    ...  
    suspend fun addLibro(libro: Libro){  
        dao.insert(libro)  
    }  
}
```

LibrosRepository.kt

Cuando una corrutina llama a una función marcada con el modificador *suspend*, en lugar de bloquearse hasta que esa función retorna como una llamada normal a una función, **suspende la ejecución hasta que el resultado está listo y entonces se reanuda donde se quedó con el resultado**.

Mientras está suspendida esperando un resultado, **desbloquea el hilo** en el que se está ejecutando **para que otras funciones o coroutines puedan ejecutarse**.

DAO con suspend

Para que las **operaciones con la base de datos** no se hagan en el hilo principal, las funciones del DAO **deben marcarse con *suspend***. Todas las funciones, **excepto las queries que devuelvan un resultado observable** (LiveData), que pueden devolver varios resultados y ya se ejecutan en segundo plano por defecto.

```
@Dao
interface LibrosDao {
    @Query("SELECT * FROM libros")
    fun getAll(): LiveData<List<Libro>>

    @Insert
    suspend fun insert(vararg libro: Libro)

    @Update
    suspend fun update(libro: Libro)

    @Delete
    suspend fun delete(libro: Libro)
}
```

LibrosDAO.kt

Lo mismo sucede con las funciones de la clase Repositorio, encargadas de interactuar con el DAO o una API remota

Dispatchers

Usar *suspend* no le dice a Kotlin que ejecute una función en un hilo de fondo. Las coroutines pueden ejecutarse en el hilo principal.

Kotlin tiene varios **dispatchers (hilos)** que puedes usar para especificar **dónde deben ejecutarse las coroutines**, dependiendo de la tarea que estés realizando:

Dispatcher	Descripción del trabajo	Ejemplo de tarea
Dispatchers.Main	UI y tareas cortas (no bloqueantes)	Actualizar LiveData, llamar a funciones de suspensión
Dispatchers.IO	Tareas de red y disco	BBDS, leer/escribir archivos
Dispatchers.Default	Uso de CPU intensivo	Parsear un JSON

Alcance o *scope*

Las corrutinas **deben ejecutarse en un CoroutineScope**:

- Mantiene un registro de todas las corrutinas iniciadas en él (incluso las suspendidas)
- Proporciona una forma de cancelar las corrutinas en el scope
- Proporciona un puente entre funciones regulares y corrutinas

Por defecto, las corrutinas se lanzan en el GlobalScope, aunque a menudo **es conveniente asociarlas al ciclo de vida del componente que la ha lanzado**. Para ello, las librerías Jetpack tienen algunos alcances definidos, por ejemplo:

- *viewModelScope* para los ViewModel
- *lifecycleScope* para las Actividades o Fragmentos

Lanzar una corrutina

Las corrutinas **se lanzan**, principalmente, **con la función *launch*** de un CoroutineScope. Lanzar **launch no es bloqueante**, por lo que sigue ejecutándose las siguientes instrucciones.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        lifecycleScope.launch {  
            coroutineFunction()  
        }  
        lifecycleScope.launch(Dispatchers.IO) {  
            coroutineFunction2()  
        }  
    }  
}
```

MainActivity.kt

```
suspend fun coroutineFunction(){  
    delay(1000L)  
    Log.d("Main", "coroutine")  
}  
  
suspend fun coroutineFunction2(){  
    CoroutineScope(Dispatchers.IO).launch {  
        Log.d("Main", "coroutine2")  
    }  
}
```

Forzar un Dispatcher en una corrutina

Si queremos que una tarea se haga en un *Dispatcher* concreto independientemente desde donde se lance (por ejemplo, escribir en una bbdd), podemos utilizar la función *withContext*. Al contrario de *launch*, esta función es una función de suspensión.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        lifecycleScope.launch {  
            coroutineFunction()  
        }  
  
        lifecycleScope.launch {  
            coroutineFunction2()  
        }  
    }  
}
```

MainActivity.kt

```
suspend fun coroutineFunction2(){  
    // En el Dispatcher donde la lancen  
    withContext(Dispatchers.IO){  
        // Cambia al Dispatcher IO  
        Log.d("Main", "coroutine2")  
    }  
    // Vuelve al dispatcher original  
}
```

Corrutinas asociadas a un ViewModel

Se define un **CoroutineScope** para cada **ViewModel** de tu aplicación. Puedes acceder a él a través de la propiedad ***viewModelScope*** del **ViewModel**.

Cualquier coroutine lanzada en este ámbito **se cancela automáticamente si se borra el ViewModel**. Las coroutines son útiles aquí para cuando tienes trabajo que necesita hacerse sólo si el **ViewModel** está activo (p.ej., leer datos de Room).

```
class MyViewModel: ViewModel() {  
  
    init {  
        viewModelScope.launch {  
            // Coroutine que se cancela cuando se borra el ViewModel  
        }  
    }  
    ....  
}
```

Ejercicio 1

Siguiendo con la aplicación de la lista de películas desarrollada en las sesiones anteriores, haz los cambios necesarios para que las operaciones sobre la base de datos se realicen en segundo plano.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>